

A Deep Dive into Blockchain-based Smart Contract-specific Security Vulnerabilities

Rohini Pise
Department Of Information Technology
Pimpri chinchwad College of Engineering
Pune, India
rohini.pise@pcceopune.org

Dr. Sonali Patil
Department Of Information Technology
Pimpri chinchwad College of Engineering
Pune, India
sonali.patil@pcceopune.org

Abstract: Blockchain smart contracts are prevalent nowadays as numerous applications are developed based on this feature. Though smart contracts are important and widely used, they contain certain vulnerabilities. This paper discusses various security issues that arise in smart contract applications. They are categorized in the smart contract platform, the applications that integrate with the Blockchain, and the vulnerabilities in smart contract code. A detailed study of smart contract-specific vulnerabilities and the defense against those vulnerabilities are presented in this article. Because of certain limitations of platforms or programming language used to write smart contract, there are possibilities of attacks on smart contracts. Hence different security measures or precautions to be taken while writing the smart contract code is discussed in this article. This will prevent the potential attacks happening on Blockchain distributed applications.

Keywords— Blockchain, Smart contract vulnerabilities, DApp.

I. INTRODUCTION:

Blockchain technology will continue to add to its popularity when more and more people learn about its numerous applications. Blockchains are used to keep track of information and verify its integrity[1]. But Blockchains can be even more powerful when the smart contracts are added to them. In this paper, the focus is given on smart contracts and their architecture. The details about various security issues typically arise in smart contract applications are discussed.

II. RELATED WORK:

A. Smart Contract:

The smart contract is a computer program stored on Blockchain and executed automatically when particular condition written in the contracts is met [2]. The smart contracts are replicated across all the miners once they are deployed on Blockchain. One feature of smart contract is that once it is deployed they can not be changed or undone. When you invoke a method, you're invoking a method across all the miners so that it can't be undone. And when you think about creating a smart contract or the method invocation of a smart contract, it is essentially represented as a transaction.

Every smart contract has three things stored in the Blockchain, specifically Ethereum.

1. It has a balance associated with it along with a public key and a private key address. User can transfer ether or money to the smart contract's address [2].

2. It also has code associated with the smart contract. That is what the miners will instantiate and run that is stored in the Blockchain.
3. And finally, it has state.

When you call methods in a smart contract, it updates the state; a block or transaction is created with that state change. It is then recorded in the Blockchain, and now all the miners will get that. They will all know about the change that has occurred with the smart contract based on invoking a method that changes the state.

B. Security implications of Smart Contracts:

1. Miners can both mine and participate as users/owners of smart contracts [3]: The important thing is to keep in mind when you're evaluating smart contract applications, Miners can both participate as users and as miners when transactions come out. They can create their own transactions to take advantage of certain situations. Create a block and if they win, submit it and get an unfair advantage. This we have to keep in mind.

2. The other thing is, when we create our applications, we create our objects, and they are usually wrapped around some web application. There is no real way for someone else to call any methods. We already know what methods will be called and when because we have written the code that calls those objects in a predetermined way. But when you deal with smart contracts, you may have an idea of how you're going to call the methods and how it's supposed to work, but the attacker can call any of these public methods in any order he wants and take advantage of this.

3. Overview of vulnerabilities in Smart contract:

Smart contracts though powerful, have specific vulnerabilities, and they fall into three groups [4].

- The smart contract platform in which the smart contracts are written,
- the applications that integrate with the Blockchain. So think about the web application that a user use it to integrate with the Blockchain and have to create transactions on that Blockchain.
- And finally, we're going to cover the smart contract code and the mistakes that developers make that cause these vulnerabilities.

Figure 1 elaborates on these categories.

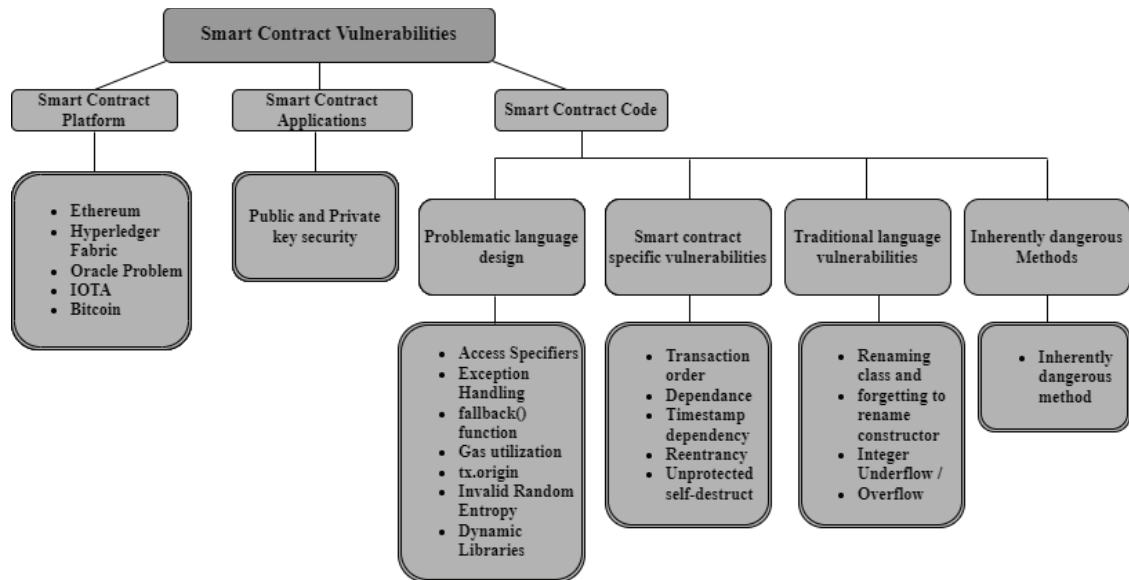


Fig. 1. Overview of Smart contract Vulnerabilities

As Shown in figure Smart Contract platforms such as Ethereum, Hyperledger Fabric, Corda etc. has certain vulnerabilities in them. These vulnerabilities are discussed in section 4. Similarly, there are vulnerabilities associated with the applications developed using Smart Contracts and these are elaborated in section 5. Lastly there are various vulnerabilities in Smart contract code including its programming language. These vulnerabilities are discussed in section 6 and 7.

C. Vulnerabilities in Smart contract platforms:

Ethereum is one of the most popular blockchain platforms with incredible features. One of them is smart contracts. On different blockchain platforms, smart contracts perform differently. Ethereum is used widely. Other platforms are Hyperledger, Corda, NEO, etc.

But the concern is, that they have security vulnerabilities. So important consideration is auditing the smart contracts for security. As Ethereum is a public platform, anyone can access any smart contract and use it for their purpose. So as a smart contract writer, we should be ready to patch.

Oracle problem is another issue with platforms. The oracle nodes are responsible for storing the critical data on Blockchain securely. The oracle nodes are a single source of truth and are susceptible to attack, compromise, manipulation, and collusion.

Another example of a platform vulnerability is IOTA, and its custom hashing algorithms. Some researchers from MIT found that there were collision issues with it. Bitcoin has Transaction malleability, and other popular platforms will be attacked as they get more activity [4].

Though Blockchains are decentralized and tamper-proof, if the attacker exploits any smart contract vulnerability, it leads to substantial financial loss.

D. Vulnerabilities in apps that integrate with a blockchain:

This section focuses on the apps that integrate with the Blockchain. Every DApp or decentralized App has some web interface to the Blockchain. And so, some vulnerabilities are associated with the web application. So make sure that you are putting together the right SDLC,

hardening the OS, reviewing your code, and controlling access to the server.

Another important thing here is the keys. When this web application signs a transaction to be put in the Blockchain, you better ensure that those private keys are appropriately protected.

E. Smart Contract Security Vulnerabilities:

The smart contract security vulnerabilities are categorized into four areas [5].

- Problematic language design, specifically in Ethereum
- Smart contract-specific vulnerabilities.
- Traditional language vulnerabilities.
- Inherently dangerous methods, like Eval in JavaScript.

F. Problematic Language Design:

In this section few features of programming language are discussed, which are to be used very carefully; otherwise, they may lead to certain attacks in smart contracts.

1) Access specifiers

Access specifiers are categorized as public, private, and default. Public specifiers mean anybody can call it, Private which means only the contract itself can call it. Other access specifiers are Internal and External. Internal, which is the contractor-derived class. External, meaning it is invokable via another contract or within this contract. Following code shows both the ways of declaring access specifiers.

```

// bad
uint x;
function bb() { // the default is public
// public code
}

// good
uint private q;
function bb() external {
}
  
```

2) Exception handling/ Error Handling:

Exceptions are thrown when:

-execution logic runs out of Gas- a certain amount of Gas is required for certain operations

- call stack reaches the limit of 1024

- the called logic invokes throw exceptions

Inconsistency of methods, throws exceptions or return Boolean to indicate that the operation succeeded successfully.

To give the examples,

- send(), call(), delegatecall() returns Boolean to indicate successful completion [5]. Send() method is used to send money. Call and delegatecall() can invoke other methods like ABI interface. They will continue with program logic when there are invocation failures. (Returns false)

Another method transfer () throws and propagate exceptions and rolls back transactions

To avoid attacks, it is safe to use the available error handling methods in the Smart Contracts. Three important functions to handle the errors in Solidity are require(), revert() and assert().

1.Require(): It is used to evaluate inputs, preconditions and outputs of functions. If not true solidity will throw an error and fail.

Example:

```
Function deposit(uint a) public {
    Uint oldB=bal;
    Uint newB=bal+a;
    Require(newB>=oldB,"Overflow"); //check the
requirement first then update the balance
    Bal += amount;
    Assert(bal>=amount;    ///this assert statement
should be true
```

2. Revert(): Is similar to require but only takes one argument which is the error message

3. Assert(): It is used to check for conditions that should never fail. Asserts should always evaluate to true.

3) Fallback () Functions:

- Syntax: function () {...}: //(functions with no name)
It does not have a name to function, no parameters. Only code written in curly braces will get executed.
- Called as default when no method matches a external request
- Called when interfaces are incorrectly specified [5]
- Called when clients send money using send(), transfer(), , call.value() etc. The fallback function is invoked whenever a send() method is called. Similarly when tranfer() function is called it inherently calls fallback() function. Also, when the interface you define is incorrectly specified, this fallback (0 function will be invoked.)
- Need payable modifier to receive money

```
// bad
function() payable { balances[msg.sender] +=
msg.value; }
```

```
// good
function deposit() payable external {
balances[msg.sender] += msg.value; }
```

```
function() payable { require(msg.data.length ==
0); emit LogDepositReceived(msg.sender); }
```

4) Gas: Running out of GAS:

Gas is needed to invoke operations, and basically, there is the cost for every function. For example, it uses five units of Gas to perform a multiply operation, three units of Gas to do addition, and all the other operations we can do have an associated cost of Gas with them. Gas is provided by an invoker of a contract method to pay for lines of code that are executed within that contract method.

So you have to make sure that you don't run out of Gas because if you run out of Gas, all the Gas that you have provided gets lost, transactions get rollback, and you lose everything [6].

Consider the following code.

```
Caddress= new address[ ] (0);
Camount= new uint[ ] (0);
```

In the above code, we are setting an address array to zero size but what it does in the EVM is it goes line by line and sets everything to zero. Now the problem comes in if that data structure has 1000 things inside it; when they go through each time each iteration instruction costs money, it will run out of Gas.

So the solution for not running out of Gas is, to make sure that you have enough Gas to execute whatever is in your loop that you need to do. And you keep track so that if you terminate the loop early and call the same function again, you can start where you left off and do the rest of the operations.

Keep a counter and take advantage of methods that do not propagate exceptions.

5) tx.origin:

Tx.origin tells who has started the transaction. So for example if A calls B (A-> B) and B calls A (B-> A), Since A calls first and started the transaction, tx.origin inside of its class will always be A[6].

It stores the address of the starting contract. tx.origin is not used for authorization [10]. The contract that uses tx.origin to authenticate can start a transaction by calling another contract. If the contract is malicious, the malicious contract can call back into the original contract, and tx.origin will be the original contract address (not the secondary contract's address). A calls->B calls-> A's vulnerable method, which checks if tx.origin==owner (which is true because A started the transaction). One of the same methods is msg.sender. So user can use msg. sender for authorization

6) Invalid Random Entropy Sources:

Some developers look for random values in the code using the following items. But these items are not

determined. So the following items are not good sources of random entropy.

Block.coinbase
Block.difficulty
Block.gaslimit
Block.number
Block.timestamp
Block.blockhash (block.number)

All of the above are invalid sources of entropy because they are known or can be set by miners.

The solution is to use valid random entropy sources. Following are some good random entropy sources:

- Utilize a commitment scheme where participants submit a hash of their random seeds.
- When it is time to use the seeds gather the seeds and verify them against the hashes
- If all hashes match the seeds provided, then hash all seeds together with block hash and use that as the random value.

Different methods used to send money:

- Address.call.value()- allows you to specify Gas, returns Boolean, and does not propagate exceptions
- Address.send(): - defaults the amount of gas to 2300. Used for contracts with simple fallback function or to send ether to people (vs. contracts). Returns true or false. No other additional logic can be used; otherwise, it will run out of Gas.
- Address.transfer() :-defaults the amount of gas to 2300 and propagates exceptions.
- All of the above methods will trigger the fallback function in the contract referenced by the "address" above.
- Use send and transfer methods only when you want to send ethers and no other additional task.

Not checking the return value from send(), call(), delegatecall() are the sources of any security issues in Ethereum. So when you are using the contract, you need to know exactly whom they call, what their code looks like, did they handle the situation appropriately.

Different methods to invoke contract methods:

1. Call():- invokes the payable method on contract, can transfer ether, and specify the amount of Gas to use
2. Delegatecall():- When you do a delegatecall to another contract, that contract runs in your context. Here you call another contract method, and whatever it does affects your stake variables. Similar to call() but runs within the caller's context. Have to be especially careful because the called contract has access to the caller's contract internals. e.g., If I do a delegatecall() to another contract and that contract sends money to another contract. It is not taking out of that contract's account; it is taking money out of my account. So when you see this delegatecall(), you have to know what they do. Because what they do is what is done to your contract.

So the question is when and why we should use delegatecall(), if this is so bad. Delegatecall() is used with libraries. When we want reusable functionality, and you look at the library's code and know what it is doing, we can use delegatecall().

3. Directly: This is another method of invoking the method.

ExternalContract.methodToInvoke()

If you use the wrong method name, many things can happen. If you are sending money, and that address does not exist, then that money is lost. Many people have tried to improve this situation, but if the address is wrong, nothing can be done.

The following can occur as a result of an error in method specification:

- If the address that the method is being invoked on is not a contract, then the call returns without errors. (If you mistype the address, Ether can be lost forever)
- If the address is of a contract and the method signature matches a method in the target contract, then that method is invoked.
- If the address is of a contract, but there was a typo in the requested method name, the fallback function is invoked. Fallback() method is a method with no name and it handles unattended invoked methods.

7) *Dynamic Libraries:*

Ensure that you understand how the libraries you are using are dynamically updated and used.

Dynamic Libraries execute as delegatecall()s. So anyone should be cautious while using the dynamic libraries.

G. *Smart contract Specific Vulnerabilities*

1) *Transaction order Dependence (TOD)*

A miner can be both; he can be a miner and a participant too. Being a owner, a miner can take advantage of transaction ordering [7].

Consider a contract as below.

```
contract Auction {
    uint public Price;
    uint public Stock;
    /.../
    Function ChangeP( uint _p) {
        If (msg.sender == owner)
            Price = _p;
    }
    Function Get (uint q) returns (uint) {
        If (msg. value < q * Price || q > Stock)
            throw;
        Stock -= q;
    }
}
```

An owner-miner can take advantage of a Get transaction. What he can do before a get transaction. He can increase the price before the get, create the block, submit it. The miner owner of the contract can see the pending get transaction in the pending transaction pool. Then create a block where he puts his update price (higher price) before the get transaction.

2) *Timestamp dependency:*

The miner can manipulate the timestamp of a transaction to exploit the vulnerability [7]. To get the benefit from the transaction, the miner can alter the timestamp by just a few seconds that best suit his profit. As a result, the contract may produce different outputs for the miner's benefit.

3) *Reentrancy:*

A reentrancy attack involves two smart contracts. A victim's contract that contains harmful code and an untrusted attacker's contract. The smart contract has to keep tracking internally, the balances of all participants [7].

Contract A calls a function in contract B

Then contract B calls back into contract A while contract A is still processing.

Contract B's fallback function is used to call back into contract A to exploit the orders of operation in a poorly written function that is tracking balances. Consider Contract A. and has the following order of transactions[10].

```
Function()
    Checkbakance
    Sendfunds
    updatebalance
```

The contract should be written in the following order. This will prevent another contract's fallback function from calling into it while processing.

```
Function()
    Checkbakance
    Updatebalance
    Sendfunds
```

4) *Unprotected self-destruct:*

Being public in nature, any user, rather any unauthorized user, also can access the smart contract incorrectly. One of the sensitive methods in smart contracts is self-destruct. Any malicious user can call the self-destruct function to destroy the contract. This can happen because of insufficient access controls [8][10].

Consider a contract:

```
Contract SimpleSuiSide {
Function suicideAnyone () {
    // unprotected selfdestruct
    Selfdestruct ( msg. sender);
}
}
```

As a defensive mechanism, self-destruct functionality can be avoided unless needed.

H. *Traditional language vulnerabilities.*

1) *Renaming class and forgetting to rename constructor:*

Sometimes developers deciding to change the class name may be because of some difficulty in naming it, but they forget to change the constructor's name. Whenever we talk about access specifiers, if there is no access specifier is defined, then, it becomes public. So now what this method will do, it sets the owner to whoever is calling it. Whoever is calling that method now becomes the owner of the contract.

If the contract is to deduct the money, the owner can do that. So be careful while changing the names of classes and constructors.

2) *Integer Underflow / Overflow:*

This is a traditional vulnerability in the smart contract. Basically, when you overflow a value, if it is an unsigned integer value when you go beyond its maximum, it wraps around and goes back to zero. When it is underflow when the value is subtracted again and again and reaches zero, and if subtracts one more it goes to the maximum value that variable can hold [9][10].

Consider example:

```
Function burnSupply (unit256 _amount) isOwner
{
    BalanceOf (owner) -= _amount;
    SupplyBurn (_amount);
}
```

Here we have a function burnSupply that a user can call to burn or delete some tokens or ethers. If one token is added and two tokens are subtracted by calling burn, for an unsigned integer of 256 bits, the number of ethers in the account will be in millions.

Defenses against Integer Underflow and Overflow:

For defending against integer underflow and overflow, you have to make sure that what you are sending is enough to be sent by the balance holder, and when you are sending it to, you have to make sure that when you add to it, it does not overflow [9].

Consider following code:

```
Function trans ( address _to, uint256 _val) {
Require ( balanceOf [ msg.sender] >= _val && balanceOf [
_to] + _val >= balanceOf [ _to]);
/* Check if the sender has balance and for overflows*/
}
```

I. *Inherently dangerous Methods.*

1) *Inherently dangerous method:*

Delegatecall(): The delegatecall() takes an ABI descriptor of a method to invoke.

Consider

```
Function() {
    Require (currentVersion.delegatecall (msg.data));
}
```

Here msg.data is a field in the transaction that is created when you push a method operation or something to the smart contract as we know that delegatecall() runs in the callers context [9][10]. Though this is considered best practice while writing smart contracts, you have to be careful. Even something that re best practices depending on the context can be abused, so you should know what it is doing.

J. *Attacks*

Attacks happened on exploiting these vulnerabilities:

Following are few attacks which occurs if the above mentioned vulnerabilities are not identified.

1. Indirect execution of unknown code: A possibility of indirect execution is there due to the presence of the fallback function feature in smart contracts.

2.Reentancy: This happens because of calling another contract in one contract.

3. Incorrect calculation of the output token amount: Because of incorrect decimals handling, incorrect order of operation during fees calculations this type of errors occurs.

4. Rubixi: Because of difference in constructor name, attacker gets control of smart contract.

III. SAFETY MEASURES:

A. Awareness of Blockchain properties

- While using external contract calls, care should be taken so that any malicious code should not get executed, and it will change the control flow.
- Being public, functions can be called maliciously, so be careful while defining public and private functions.
- Be aware of the gas limit and price while executing the transactions.

B. Keep it Simple

- Use simple logic to write the contracts, also prefer modular programming.
- Analyze whether your application requires blockchain functionality for decentralization, if yes then only use it.

C. Awareness of the latest security patches:

- Keep on discovering new bugs and new security techniques in contracts

Upgrade to the latest version of any tool or library as soon as possible.

IV. CONCLUSION:

Smart contracts and Ethereum are new technologies facing new security challenges. Hence more research on security mechanisms is needed. As new bugs and security risks are discovered, the focus should be on the development of best practices. By considering the need, this article is prepared to focus on various vulnerabilities specific to smart contracts and security defenses to avoid these vulnerabilities.

REFERENCES

- [1] Bhattacharya P, Singh A , "A Systematic Review on Evolution of Blockchain Generations ITEE Journal A Systematic Review on Evolution of Blockchain Generations", International Journal of Information Technology and Electrical Engineering, December 2018.
- [2] Ilya Grishchenko, "A Semantic Framework for the Security Analysis of Ethereum smart contracts", 10.1007/978-3-319-89722-6_10, 2018
- [3] Rouhani, Sara Rouhani, "Security, performance, and applications of smart contracts: A systematic survey", 10.1109/ACCESS.2019.2911031, IEEE Access 2019
- [4] Praitheeshan, Purathani, Pan, Lei, "Security Analysis Methods on Ethereum Smart Contract Vulnerabilities: A Survey", ArxivID 1908.08605, 2019
- [5] "Smart Contract Privacy Protection Using AI in Cyber-Physical Systems: Tools, Techniques and Challenges", IEEE Access, 24746-24772, 8, SPECIAL SECTION ON CLOUD - FOG - EDGE COMPUTING IN CYBER-PHYSICAL-SOCIALSYSTEMS (CPSS), 2020
- [6] Atzei, Nicola, Bartoletti, Massimo, "A survey of attacks on Ethereum smart contracts", 2018.
- [7] Liu J, Liu Z, "A Survey on Security Verification of Blockchain Smart Contracts", ACCESS.2019.2921624 , IEEE Access 2019
- [8] Cong, Lin William, He, Zhiguo, "Blockchain Disruption and Smart Contracts", Review of Financial Studies, 2019.
- [9] Sun, Tianyu Yu, Wensheng, "A formal verification framework for security issues of blockchain smart contracts", Electronics (Switzerland), 10.3390/electronics9020255, 2020
- [10] <https://consensys.github.io/>
- [11] Rajvardhan Oak, Karanveer Singh Jhala, Mrunmayee Khare, "Smart Collaboration Mechanism using Blockchain Technology", 2018 5th IEEE International Conference on Cyber Security and Cloud Computing (CSCloud) 2018